

AI Agents That Work & Earn on Mantle

Token Standard · User Flow · Features · Build Guide — Mantle Turing Test Hackathon 2026

1. TOKEN STANDARD — ERC-8004 (TRUSTLESS AGENTS)

ERC-8004 is confirmed as the official standard across all Mantle hackathon announcements. But it is critical to understand what it actually is — most teams will get this wrong.

ERC-8004 is NOT a Simple NFT Standard:

ERC-8004 is a trust and identity layer for autonomous AI agents. It extends the Agent-to-Agent (A2A) protocol with three lightweight on-chain registries that allow agents to be discovered, trusted, and interacted with across organizational boundaries without pre-existing trust. The three registries are:

Identity Registry	Records who the agent is — owner address, agent name, endpoint URL, creation timestamp, current status. This is the agent's permanent on-chain passport. Every Spawn agent gets registered here on creation.
Reputation Registry	Records what the agent has done — every task completion, action executed, outcome, and value. In Spawn, every time an agent executes an on-chain action it logs a reputation event here. This is the core of Spawn's transparency.
Validation Registry	Records third-party attestations about the agent. For Spawn's MVP this is implemented as a minimal stub — but including it shows judges you understand the full standard, not just the NFT part.

Most teams will mint a regular ERC-721 NFT and call it ERC-8004. Spawn implements all three registries correctly and uses the Reputation Registry for every agent action. This is the single biggest technical differentiator in the submission.

Full spec: eips.ethereum.org/EIPS/eip-8004 — read before writing any smart contracts.

2. WHAT SPAWN DOES — COMPLETE FEATURE LIST

Every feature is scoped to be buildable in 6 weeks with AI-assisted development. Nothing is vague — each feature maps to a specific build task.

FEATURE 1 — Natural Language Agent Creation

What it does	User types what they want their agent to do in plain English. The system parses this into a structured agent config: goals, triggers, conditions, and on-chain actions.
Example input	"Check my mETH yield every day. If it drops below 4%, alert me on Telegram and move my funds to Agni Finance automatically."
What it outputs	A JSON config: { goal, trigger_type, trigger_value, action_type, action_params, alert_channel, frequency, agent_type }
How to build	Claude API with a strict JSON-output system prompt. Input = user description. Output = validated agent config. Add re-prompt logic if required fields are missing.
Why judges care	This is the consumer magic. Non-technical users creating powerful on-chain agents without any Web3 knowledge. Animoca and Virtuals judges will immediately see the mass-market potential.

FEATURE 2 — Agent Wallet (On-Chain Treasury)

What it does	Every Spawn agent gets its own dedicated smart contract wallet on Mantle. The user funds this wallet with mETH or USDY. The agent uses this budget to pay gas and execute transactions.
Why mETH/USDY	mETH earns liquid staking yield while idle — the agent's budget is always productive. USDY earns T-bill yield for conservative agents. The operating capital never just sits there.
Smart contract logic	AgentWallet.sol: stores owner, allowed actions whitelist, max spend per transaction, balance, pause() function for owner.
Security model	Owner sets max spend per transaction. Owner can pause or withdraw at any time. Agent cannot exceed its defined action whitelist. These controls are essential for user trust.
How to build	Factory pattern: SpawnFactory.sol deploys a new AgentWallet for each agent and registers it in the ERC-8004 Identity Registry in one transaction.

FEATURE 3 — ERC-8004 Identity & Reputation

What it does	On agent creation: registers agent in Identity Registry. On every completed action: logs a Reputation Registry entry with timestamp, action type, outcome, and value transferred.
Identity stores	Agent name, owner address, agent type, creation timestamp, current status (active/paused/retired).

Reputation stores	Action type, trigger condition, action executed, outcome (success/fail), value amount, block number, timestamp.
Why it matters	This is the hackathon's core standard. Implementing it fully — not just minting a token — is what separates serious submissions from superficial ones.
How to build	Three Solidity contracts: IdentityRegistry.sol, ReputationRegistry.sol, ValidationRegistry.sol. Deployed once. All agents share the same registries.

FEATURE 4 — Three Demo Agents

Yield Guardian	Polls mETH yield from Agni Finance every 4 hours. If yield drops below user threshold: logs reputation event + sends Telegram alert + optionally moves funds.
DCA Agent	On a user-set schedule, converts a fixed amount of USDY to mETH via Merchant Moe. Builds mETH position passively. Logs every swap to Reputation Registry.
Wallet Sentinel	Monitors user's Mantle wallet for outgoing transactions above a set threshold. Fires instant Telegram alert. Logs all monitoring activity to Reputation Registry.
Shared architecture	All three use the same ERC-8004 standard, same agent wallet, same dashboard display, same Telegram system. Build once, run three times.

FEATURE 5 — Agent Dashboard

What it shows	Agent name, type, live status. Current wallet balance. Yield earned on idle budget. Total actions executed. Full on-chain activity log.
Activity log	Pulls Reputation Registry events for this agent. Shows timestamp, action, trigger condition, outcome, transaction hash — all clickable to Mantle Explorer.
Controls	Fund agent (add mETH/USDY). Pause agent. Modify trigger thresholds. Retire agent (returns remaining budget to owner wallet).
How to build	Next.js frontend. Read on-chain data via viem. Reputation Registry events displayed as a live feed. Clean consumer UI — not a developer tool.

FEATURE 6 — Telegram Alerts

What it does	When an agent trigger fires, sends a formatted Telegram message to the user's configured chat.
Message format	■ [Agent Name] fired Trigger: mETH yield dropped to 3.8% Action: Moved 0.5 mETH to Agni Finance Tx: 0xabc...
How to build	Telegram Bot API. User connects Telegram during onboarding (/start to bot). Agent runtime calls sendMessage on every trigger. No extra infrastructure needed.

3. COMPLETE USER FLOW — SCREEN BY SCREEN

This is the exact journey from landing page to a live working agent. Build in this exact order — it is also the demo script for judges.

1	Landing Page	Headline: 'Hire an AI agent. Fund it with mETH. Watch it work.' Single CTA: 'Spawn Your First Agent'. No signup. Connect wallet to start.
2	Connect Wallet	MetaMask or WalletConnect. App detects Mantle network. Prompts network switch if needed. Shows wallet address + mETH/USDY balance once connected.
3	Choose Agent Type	Three cards: Yield Guardian, DCA Agent, Wallet Sentinel. Each shows name, one-line description, estimated monthly gas cost, example use case. Plus 'Custom' option for power users.
4	Configure Agent	Simple form for the chosen agent type. Yield Guardian: yield threshold % + auto-move toggle. DCA: amount + frequency. Sentinel: alert threshold. No Web3 jargon anywhere.
5	Name Your Agent	User gives the agent a name. Stored permanently in ERC-8004 Identity Registry on Mantle. Optional avatar image stored as IPFS URL in identity metadata.
6	Fund Your Agent	Input: amount of mETH or USDY to deposit. Shows estimated monthly operating cost. Shows: 'Your idle balance earns X% APY while agent is active.' User approves token transfer.
7	Agent Deployed	Success screen. ERC-8004 identity minted (with Mantle Explorer link). Agent wallet address shown. Telegram connect button. 'View Dashboard' CTA.
8	Dashboard	All user agents listed. For each: status indicator, current budget, yield earned, total actions, last action time. Click any agent to open its full activity log.
9	Activity Log	Every trigger that fired. Every action executed. Every outcome. Pulled from ERC-8004 Reputation Registry on Mantle. Transaction hash links to Mantle Explorer.
10	Manage Agent	Top up wallet. Pause agent. Modify thresholds. Retire agent (withdraws remaining budget). Full owner control at all times.

4. REALISTIC BUILD TIMELINE — AI-ASSISTED DEVELOPMENT

This timeline is scoped for a vibe coder using AI tools (Claude, Cursor, Copilot) to accelerate development. AI-assisted coding changes the pace significantly — what would take a traditional developer 3 weeks can be compressed into 1 week when you use AI to generate boilerplate, debug errors, and scaffold components. The timeline reflects this honestly.

Total build time with AI-assisted coding: 5–6 weeks. Each week has one primary deliverable. The goal at end of each week is a working demo of that piece — not perfect code, working product.

Week 1	May 1–7	Foundation + ERC-8004 Contracts
• Set up Mantle dev environment — Node.js, viem, Hardhat • Deploy ERC-8004 Identity, Reputation, Validation registry contracts on Mantle testnet • Build Claude API → agent config JSON parser • Scaffold Next.js frontend with wallet connect (wagmi) • END OF WEEK: wallet connects, agent description generates valid config, identity contract deployed ✓		
Week 2	May 8–14	Agent Wallet + Runtime Core
• Write and deploy AgentWallet.sol + SpawnFactory.sol on testnet • Build Node.js agent runtime loop — reads config, executes checks on schedule • Connect runtime to Mantle RPC via viem — read wallet balances and DeFi data • Implement Yield Guardian logic end-to-end on testnet • END OF WEEK: first working agent deployed on testnet, fires and logs to Reputation Registry ✓		
Week 3	May 15–21	DeFi Integration + Telegram
• Integrate Agni Finance read operations — fetch mETH yield rates • Integrate Merchant Moe — implement USDY → mETH swap for DCA Agent • Build Telegram bot — connect user, send alerts on trigger • Build agent funding flow on frontend (deposit mETH/USDY to agent wallet) • END OF WEEK: agent with funded wallet executes real DeFi action, Telegram alert received ✓		
Week 4	May 22–28	Dashboard + All Three Agents

- Build full agent dashboard — activity log from Reputation Registry, budget display, controls
- Implement DCA Agent and Wallet Sentinel logic
- Deploy to Mantle mainnet — test with real small amounts
- Wire complete frontend flow: landing → configure → fund → deploy → dashboard
- END OF WEEK: all three agents working on Mantle mainnet ✓

**Week
5**

May 29 –
Jun 4

Polish + Security

- UI/UX polish — consumer-grade look and feel, not a developer tool
- Security review: agent wallet spend limits, owner withdrawal, action whitelist
- Test all three agents simultaneously with real mETH and USDY
- Record demo video showing full user flow on mainnet
- END OF WEEK: demo-ready product a real person could use ✓

**Week
6**

Jun 5–15

Submission

- Final bug fixes based on testing
- Write DoraHacks submission — lead with 'why Mantle', show all three agents live
- Push all code to public GitHub with clean README
- Submit before June 15 — do not wait until the last day
- END OF WEEK: submitted ✓

5. HOW TO BUILD SPAWN — TECHNICAL GUIDE

Key code patterns for each major component. Use these as starting points — AI coding tools can scaffold the full implementation from these skeletons.

ERC-8004 Contracts (Solidity)

```
// IdentityRegistry.sol
contract IdentityRegistry {
    struct AgentIdentity {
        address owner;
        string name;
        string agentType;
        string endpoint;
        uint256 createdAt;
        bool active;
    }
    mapping(bytes32 => AgentIdentity) public agents;
    event AgentRegistered(bytes32 agentId, address owner, string name);
    function registerAgent(string memory name, string memory agentType, string memory endpoint) external returns (bytes32 agentId) {
        agentId = keccak256(abi.encodePacked(msg.sender, name, block.timestamp));
        agents[agentId] = AgentIdentity(msg.sender, name, agentType, endpoint, block.timestamp, true);
        emit AgentRegistered(agentId, msg.sender, name);
    }
}

// ReputationRegistry.sol
contract ReputationRegistry {
    struct ReputationEvent {
        bytes32 agentId;
        string actionType;
        string outcome;
        uint256 value;
        uint256 timestamp;
    }
    mapping(bytes32 => ReputationEvent[]) public agentHistory;
    event ReputationLogged(bytes32 agentId, string actionType, string outcome);
    function logAction(bytes32 agentId, string memory actionType, string memory outcome, uint256 value) external {
        agentHistory[agentId].push(ReputationEvent(agentId, actionType, outcome, value, block.timestamp));
        emit ReputationLogged(agentId, actionType, outcome);
    }
}
```

■ Keep these contracts lean — every agent action calls `logAction()` which costs gas. The leaner the contract, the cheaper each reputation log. Prompt your AI coding tool: 'Write a minimal Solidity ERC-8004 Identity Registry with owner, name, agentType, endpoint, createdAt fields and a registerAgent function.'

Claude API Config Parser (Node.js)

```
const parseAgentDescription = async (userDescription) => {
    const response = await fetch("https://api.anthropic.com/v1/messages", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            model: "claude-sonnet-4-20250514",
            max_tokens: 500,
            system: `You are an agent config parser. Output ONLY valid JSON. Required fields: goal (string), trigger_type (price_threshold|schedule|wallet_event), trigger_value (number or cron string), action_type (alert|swap|move_funds|monitor), action_params (object), alert_channel (telegram|none), frequency (string), agent_type (yield_guardian|dca_agent|wallet_sentinel|custom). Never include explanations. Output the JSON object only.`,
            messages: [{ role: "user", content: userDescription }]
        })
    });
    const data = await response.json();
    return JSON.parse(data.content[0].text);
};
```

■ The system prompt is everything here. Be extremely specific about the JSON schema. Test with 20 different user descriptions before moving on. Add validation that checks for required fields and re-prompts automatically if any are missing.

Agent Runtime Loop (Node.js)

```
const runAgent = async (agentConfig, agentWalletAddress, agentId) => { while
(agentConfig.status === 'active') { try { const triggered = await checkTrigger(agentConfig);
if (triggered) { const result = await executeAction(agentConfig, agentWalletAddress); await
logToReputationRegistry(agentId, agentConfig.action_type, result); await
sendTelegramAlert(agentConfig.telegramChatId, result); } } catch (err) { await
logError(agentId, err); } await sleep(agentConfig.frequency_ms); } };
```

■ Each agent runs as its own async loop. For MVP, run all loops on one Node.js server using a simple map of agentId → running loop. Prompt AI: 'Write a Node.js agent runtime that reads a config object, checks a trigger condition via RPC, executes an action, and logs to a smart contract.'

Mantle RPC + DeFi Reads (viem)

```
import { createPublicClient, http } from 'viem' import { mantle } from 'viem/chains' const
client = createPublicClient({ chain: mantle, transport: http('https://rpc.mantle.xyz') }) //
Read wallet balance const balance = await client.getBalance({ address: userAddress }) // Read
mETH balance const mEthBalance = await client.readContract({ address: METH_CONTRACT_ADDRESS,
abi: mEthAbi, functionName: 'balanceOf', args: [userAddress] }) // For Agni Finance yield –
check their docs for correct function name // likely getPoolData() on their factory contract
```

■ Always use viem over ethers.js for new projects. Get all contract addresses from Mantle's official documentation. Prompt AI: 'Write a viem client that connects to Mantle mainnet and reads the mETH balance and Agni Finance yield rate for a given wallet address.'

6. HOW TO PITCH SPAWN — SUBMISSION GUIDE

"Most people have money sitting idle on-chain. Spawn gives that money a job."

Five Points Your Submission Must Hit:

THE PROBLEM	Managing DeFi positions manually is impossible for regular people. They either do nothing with their on-chain assets or rely on expensive fund managers.
WHY MANTLE	Mantle's \$0.002 fees make micro-agent operations economically viable for the first time. mETH and USDY mean the agent's budget earns yield while it works. No other chain makes this combination possible.
THE PRODUCT	Spawn: describe your agent in plain English, fund it with mETH or USDY, it executes on Mantle autonomously with full on-chain transparency via ERC-8004.
THE DEMO	Show all three agents live on mainnet. Show ERC-8004 identity minted. Show Reputation Registry activity log. Show value flowing back to user wallet.
THE VISION	Spawn is the consumer entry point to Mantle's DeFi ecosystem — the product that brings regular people on-chain without requiring them to understand Web3.

Spawn · Primary Submission · Mantle Turing Test Hackathon 2026 · Consumer DApps Track · Token Standard: ERC-8004 · eips.ethereum.org/EIPS/eip-8004 · Deadline: June 15 2026